

Advanced Machine Learning

T.A Merit Samir



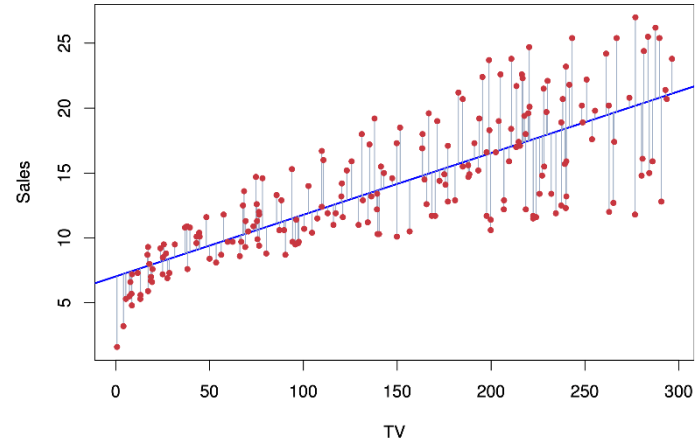
Lab 5

1. Regression .
2. Linear Regression .
3. Polynomial Regression.
4. Regularized Linear Models
 - a.Ridge Regression (L2 Regularization)
 - b.Lasso Regression (L1 Regularization)
 - c.Elastic Net (Combination of L1 & L2)
5. Task 5



What is Regression?

- Regression predicts a continuous value (e.g., price, temperature).
- Linear regression fits a straight line: $y = a + bX$.
- Works well only if data is linear.



Linear Regression (Basic Example)

Linear regression tries to fit the **best straight line** through the data.

Equation:

$$y = a + bX$$

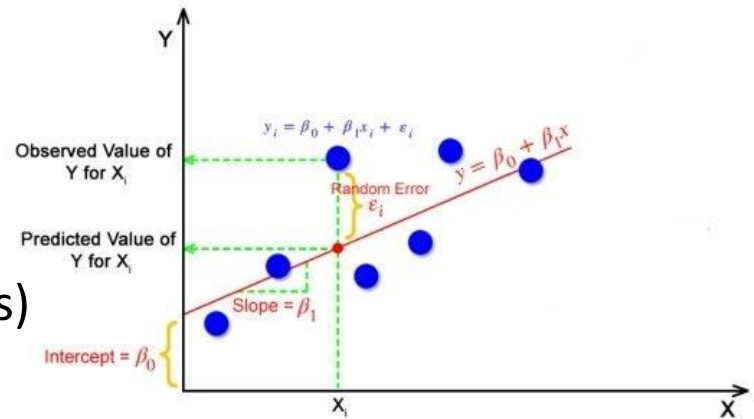
Where:

y = predicted output

X = input feature

a = intercept (where the line crosses y-axis)

b = slope (how much y changes when X increases)



Code Example

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

# 1. Generate simple data (line with some noise)
np.random.seed(42)
X = np.random.rand(50, 1) * 10      # random values between 0 and 10
y = 3 * X + 5 + np.random.randn(50, 1) * 2  # y = 3x + 5 + noise
```

```
# 2. Create and train the model
model = LinearRegression()
model.fit(X, y)
```

```
LinearRegression
```

```
# 3. Make predictions
X_new = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = model.predict(X_new)

# 2. Create and train the model
model = LinearRegression()
model.fit(X, y)
```



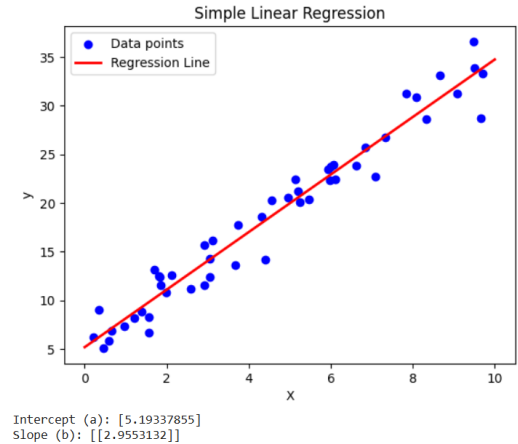
Code Example

```
# 3. Make predictions
X_new = np.linspace(0, 10, 100).reshape(-1, 1)
y_pred = model.predict(X_new)

# 2. Create and train the model
model = LinearRegression()
model.fit(X, y)

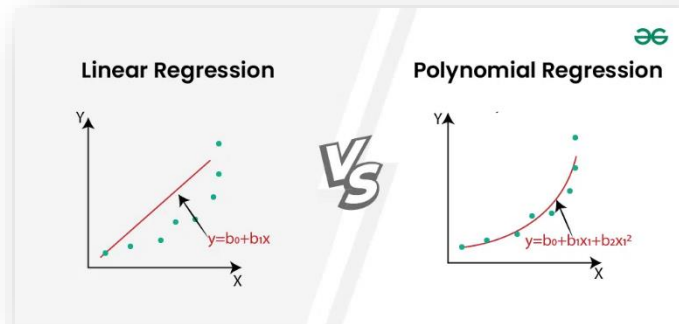
# 4. Show the line and the data
plt.scatter(X, y, color='blue', label='Data points')
plt.plot(X_new, y_pred, color='red', linewidth=2, label='Regression Line')
plt.title("Simple Linear Regression")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()

# 5. Print coefficients
print("Intercept (a):", model.intercept_)
print("Slope (b):", model.coef_)
```



Polynomial Regression

- **Idea:** Extend linear regression by adding powers of the input feature (X^2 , X^3 , ..).
- **Goal:** Allow the model to **fit non-linear** (curved) relationships.
- **Equation:** $y = a + b_1X + b_2X^2 + b_3X^3 + \dots$
- **Note:** Still a *linear model* in parameters (a , b_1 , b_2 ...), but nonlinear in X .
- **Main issue:** Can easily **overfit** when the degree is high.



Code Example

Extends linear regression by adding powers of X.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

# 1. Generate nonlinear data (not a straight line)
np.random.seed(42)
X = np.random.rand(50, 1) * 10
y = 2 + 1.5 * X + 0.5 * X**2 + np.random.randn(50, 1) * 3 # quadratic relation

# 2. Transform the data into polynomial features (degree = 2)
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)

# 3. Train the model on polynomial features
model = LinearRegression()
model.fit(X_poly, y)
```

```
LinearRegression
LinearRegression()
```



Code Example

4. Prepare new data for prediction

```
X_new = np.linspace(0, 10, 100).reshape(-1, 1)
```

```
X_new_poly = poly.transform(X_new)
```

```
y_pred = model.predict(X_new_poly)
```

5. Plot the results

```
plt.scatter(X, y, color='blue', label='Data points')
```

```
plt.plot(X_new, y_pred, color='red', linewidth=2, label='Polynomial Regression Curve')
```

```
plt.title("Polynomial Regression (Degree = 2)")
```

```
plt.xlabel("X")
```

```
plt.ylabel("y")
```

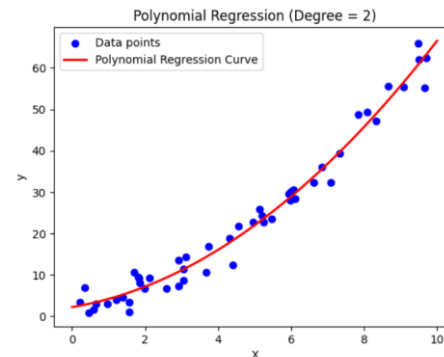
```
plt.legend()
```

```
plt.show()
```

6. Show model coefficients

```
print("Coefficients:", model.coef_)
```

```
print("Intercept:", model.intercept_)
```



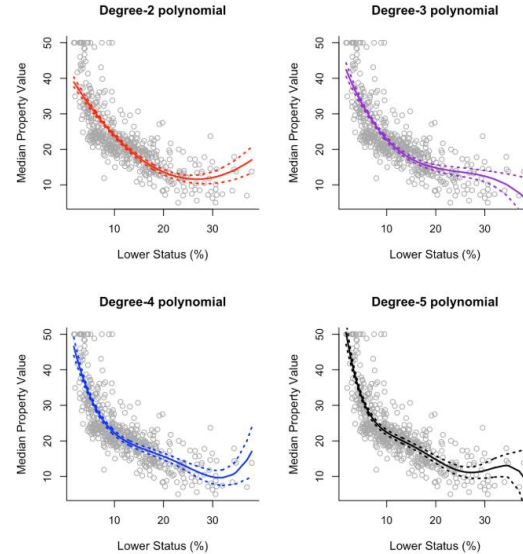
Coefficients: $[[1.47305505 \ 0.49589025]]$
Intercept: $[2.22665119]$



Polynomial Regression

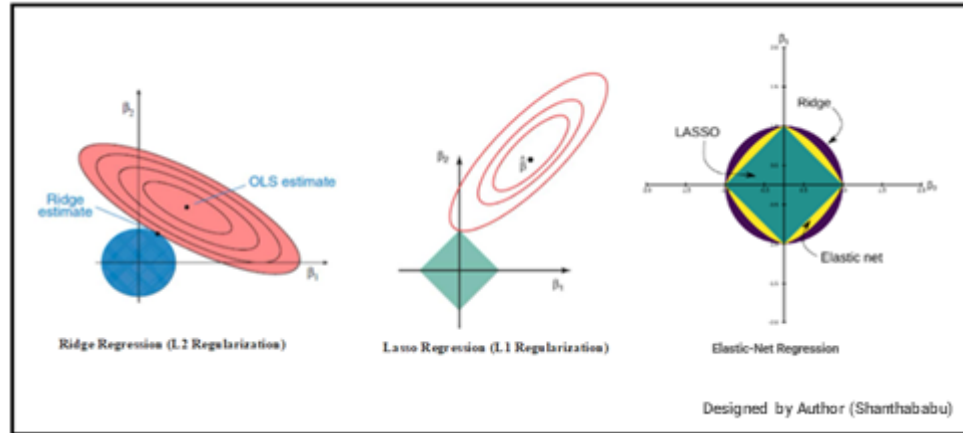
Problem: Overfitting

- High-degree polynomials can overfit (memorize data).
- Regularization adds a penalty to large coefficients.
- Helps control model complexity.



Regularized Linear Models

- These models are used to **reduce overfitting** by **penalizing large coefficients**.
- They still use a *linear equation*, but they add a penalty term to the cost function.



What Are Coefficients?

- In regression, **coefficients** are the numbers that show **how much each feature (x)** affects the **output (y)**.

- Example:**

$$y = 2x + 3$$

2 → coefficient (slope)

→ For every +1 increase in x, y increases by +2

3 → intercept (the constant term)

In Code:

```
python

from sklearn.linear_model import LinearRegression
model = LinearRegression()
model.fit(X, y)
print(model.coef_) # coefficients
print(model.intercept_) # constant term
```

Meaning:

If your model predicts house price:

$$price = 1500 \times area + 20000$$

- 1500** = coefficient → price increases by 1500 for every extra m²
- 20000** = intercept → base price when area = 0



a. Ridge Regression (L2 Regularization)

Penalty: Adds the *sum of squares* of the coefficients.

Effect: Shrinks coefficients toward zero (makes them smaller but never exactly zero).

Good for: When all features are useful but you want to reduce their influence slightly.

Equation:

$$\text{Loss} = \text{RSS} + \alpha \sum b_j^2$$



RSS (Residual Sum of Squares)

Definition:

RSS measures how far the predicted values are from the actual data. $RSS = \sum (y_i - \hat{y}_i)^2$

It's the **sum of squared errors** between predicted and true values.

Goal:

Linear regression tries to **minimize RSS** to find the best-fitting line.

In short:

Smaller RSS → better model fit.



a. Ridge Regression (L2 Regularization)

المفهوم	المعادلة	الهدف
Prediction (Output)	$b + Xw = \hat{y}$	يستخدمها النموذج لإخراج النتيجة
Learning (Training)	$\alpha \sum w^2 + \sum (y - \hat{y})^2 = J(w)$	يستخدمها لتحديث القيم w, b حتى تكون $\hat{y}$ قريبة من y

تدخل $X \rightarrow$ النموذج يطلع $\hat{y}$ (predicted y)
يقارنها بـ y الحقيقي $\rightarrow$ يحسب $J(w) \rightarrow$ يعدّل نفسه لحد ما الخطأ يصغر جدًا 

Code Example

```
# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Ridge
from sklearn.model_selection import train_test_split

# 2. Generate simple linear data
np.random.seed(42)
X = np.random.rand(50, 1) * 10          # Random X values between 0 and 10
y = 3 * X + 5 + np.random.randn(50, 1) * 2  # y = 3x + 5 + some noise

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Create Ridge model with regularization strength alpha
ridge_model = Ridge(alpha=1.0)  # alpha controls the amount of regularization
ridge_model.fit(X_train, y_train)

# 5. Predict values using the trained model
y_pred = ridge_model.predict(X_test)
```



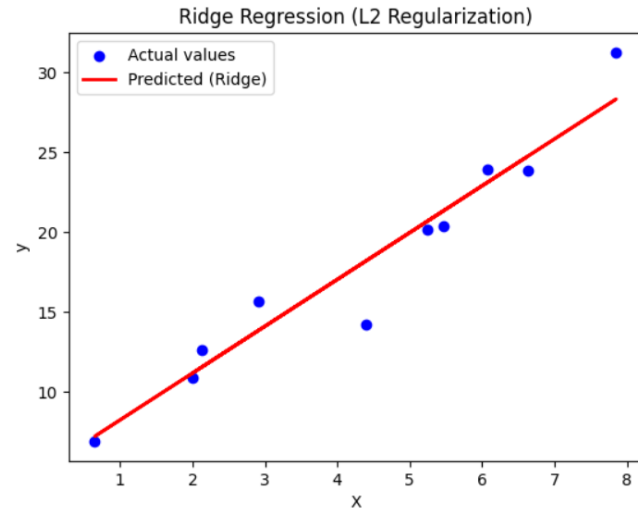
Code Example

6. Plot actual vs predicted values

```
plt.scatter(X_test, y_test, color='blue', label='Actual values')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted (Ridge)')
plt.title("Ridge Regression (L2 Regularization)")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

7. Print model parameters

```
print("Intercept:", ridge_model.intercept_)
print("Coefficient:", ridge_model.coef_)
```



Intercept: [5.28396874]
Coefficient: [2.93658094]



b. Lasso Regression (L1 Regularization)

Penalty: Adds the *sum of absolute values* of the coefficients.

Effect: Some coefficients become **exactly zero**, which means **automatic feature selection**.

Good for: When you think only a few features are important.

Equation:

$$\text{Loss} = \text{RSS} + \alpha \sum |b_j|$$



Code Example

```
# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import Lasso
from sklearn.model_selection import train_test_split

# 2. Generate sample data
np.random.seed(42)
X = np.random.rand(50, 1) * 10      # Random X values between 0 and 10
y = 3 * X + 5 + np.random.randn(50, 1) * 2  # y = 3x + 5 + noise

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Create Lasso model with regularization strength alpha
lasso_model = Lasso(alpha=0.5)  # alpha controls the amount of regularization
lasso_model.fit(X_train, y_train)

# 5. Predict values using the trained model
y_pred = lasso_model.predict(X_test)
```



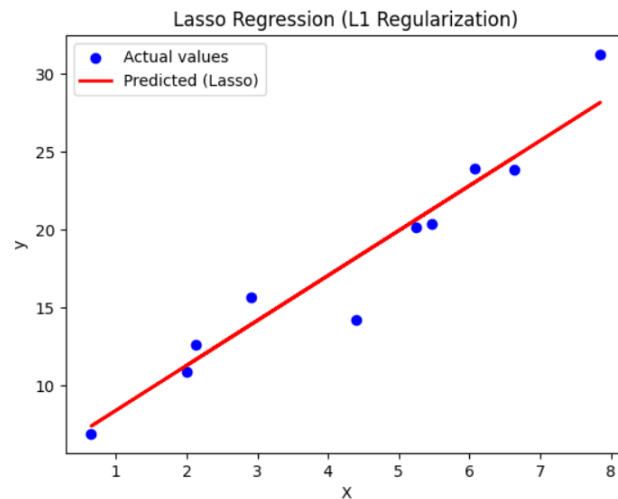
Code Example

6. Plot actual vs predicted values

```
plt.scatter(X_test, y_test, color='blue', label='Actual values')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted (Lasso)')
plt.title("Lasso Regression (L1 Regularization)")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

7. Print model parameters

```
print("Intercept:", lasso_model.intercept_)
print("Coefficient:", lasso_model.coef_)
```



Intercept: [5.49679344]
Coefficient: [2.88918397]



b. Lasso Regression (L1 Regularization)

المفهوم	الشرح
L1 Regularization	.Adds a penalty proportional to the absolute value of coefficients
Equation	$J(w) = \sum (y_i - \hat{y}_i)^2 + \alpha \sum w $
Goal	Reduce overfitting and make some coefficients exactly zero (feature selection)
Difference from Ridge	Ridge shrinks all coefficients, but Lasso can eliminate some features completely

Quick Comparison 🏆		
Effect	Penalty	Type
Shrinks coefficients smoothly (no zeros)	$\alpha \sum w^2$	Ridge
Shrinks coefficients to zero (feature selection)	$\alpha \sum w $	Lasso



c. Elastic Net (Combination of L1 & L2)

Penalty: A mix between Ridge and Lasso penalties.

Effect: Keeps stability from Ridge and sparsity from Lasso.

Good for: Datasets with **many correlated features**.

Equation:

$$\text{Loss} = \text{RSS} + \alpha \left[(1 - r) \sum b_j^2 / 2 + r \sum |b_j| \right]$$

where $r = l1_ratio$ controls the balance between L1 and L2.



Code Example

```
# 1. Import libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import ElasticNet
from sklearn.model_selection import train_test_split

# 2. Generate sample data
np.random.seed(42)
X = np.random.rand(50, 1) * 10          # Random X values between 0 and 10
y = 3 * X + 5 + np.random.randn(50, 1) * 2  # y = 3x + 5 + some noise

# 3. Split data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 4. Create Elastic Net model
# alpha → controls total regularization strength
# l1_ratio → controls balance between L1 (Lasso) and L2 (Ridge)
elastic_model = ElasticNet(alpha=0.5, l1_ratio=0.5)
elastic_model.fit(X_train, y_train)

# 5. Predict values using the trained model
y_pred = elastic_model.predict(X_test)
```



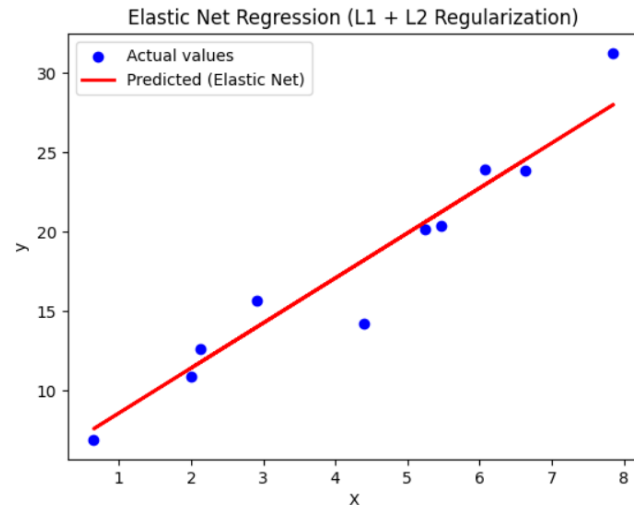
Code Example

6. Plot actual vs predicted values

```
plt.scatter(X_test, y_test, color='blue', label='Actual values')
plt.plot(X_test, y_pred, color='red', linewidth=2, label='Predicted (Elastic Net)')
plt.title("Elastic Net Regression (L1 + L2 Regularization)")
plt.xlabel("X")
plt.ylabel("y")
plt.legend()
plt.show()
```

7. Print model parameters

```
print("Intercept:", elastic_model.intercept_)
print("Coefficient:", elastic_model.coef_)
```



Intercept: [5.72605506]
Coefficient: [2.83812643]



c. Elastic Net (Combination of L1 & L2)

Explanation	Concept
.Combines both L1 (Lasso) and L2 (Ridge) regularization terms	Elastic Net
$J(w) = \sum (y_i - \hat{y}_i)^2 + \alpha[(1 - r)\sum w_j^2 + r\sum w_j]$	Equation
.Controls how strong the overall penalty is	α (alpha)
.Controls how much of the penalty comes from L1 vs L2 $r = 1 \rightarrow$ acts like Lasso $\rightarrow$ $r = 0 \rightarrow$ acts like Ridge $\rightarrow$	r (ratio)
.Reduces overfitting and can zero out some features. Balances both methods	Purpose



Comparison Between Linear, Polynomial, and Regularized Regression

Regularized Regression (Ridge / Lasso)	Polynomial Regression	Linear Regression	Feature / Type
Usually straight, but more stable	Curved (nonlinear)	Straight line	Shape of Relationship
penalty term + $bX + a = y$	$\dots + {}^2b_2X + b_1X + a = y$	$bX + a = y$	Model Equation
Same as Linear, but with penalty on coefficients	.Adds X^2, X^3 , etc	Uses X only	Extra Features
Prevent overfitting and stabilize model	Fit curves using polynomial terms	Find the best straight line	Purpose
Moderate — balances simplicity & accuracy	More complex (more features)	Simple	Complexity
Low — penalty reduces overfitting	High (if degree too large)	Low (if data is truly linear)	Risk of Overfitting
Adds a regularization term (L1 or L2)	Polynomial feature expansion	Least squares	Key Technique
Improves generalization and stability	Captures curved relationships	Simple, fast, easy to interpret	Pros
May shrink coefficients too much	Can easily overfit	Can't fit nonlinear data	Cons
Avoiding overfitting in complex models	Modeling curved patterns (e.g., growth rate)	Predicting linear trends (e.g., sales vs. time)	Example Use Case



Task 4

In this task, you will apply **various regression models** to a **real-world dataset** downloaded from **Kaggle**.

You will train, visualize, and compare **Linear, Polynomial, Ridge, Lasso, and Elastic Net Regression** models.

The goal is to understand how regularization affects model performance and complexity.

- Choose **one dataset suitable for regression**, such as: *House Prices — Advanced Regression Techniques*

<https://www.kaggle.com/competitions/house-prices-advanced-regression-techniques>

- OR any dataset with **numerical target variable** (like predicting prices, scores, etc.)





merit.samir@nmu.edu.eg

Thank you

